

Vibe Coding 工作坊

课前准备材料

目录

- [课前准备](#)
 - [前置要求](#)
 - [步骤 1: 安装 Qwen Code CLI](#)
 - [步骤 2: 配置 API 平台与 Key](#)
 - [步骤 3: 验证安装](#)
 - [步骤 4: 配置 Python 环境 \(推荐\)](#)
- [知识介绍](#)
 - [2.1 LLM 基础](#)
 - [2.2 Prompt Engineering](#)
 - [2.3 Vibe Coding](#)
 - [2.4 Harness Engineering](#)
 - [2.5 Context \(上下文\) 简介](#)

课前准备

前置要求

- macOS 或 WSL (Ubuntu)
- 对应的 VS Code
- 已安装 Git, conda
- 基础 Python 知识

步骤 1: 安装 Qwen Code CLI

```
# 推荐: 使用官方安装脚本 (会自动安装 Node.js)
bash -c "$(curl -fsSL https://qwen-code-assets.oss-cn-hangzhou.aliyuncs.com/installation/install-qwen.sh)"

# 备用: 使用 Homebrew (macOS)
brew install qwen-code

# 备用: 使用 npm (需要 Node.js 20+)
npm install -g @qwen-code/qwen-code@latest
```

注意: 官方安装脚本会自动安装所需的 Node.js 版本, 无需手动安装。

步骤 2: 配置 API 平台与 Key

说明：Qwen Code 现已不再支持免费的 OAuth 登录模式，需要使用 API Key 方式访问大模型。

2.1 选择 API 平台

推荐使用 **Xi-API** 平台：

- **官网：**<https://api.xi-ai.cn/>
- **价格：**充 10 元左右即可使用较长时间

其他可选平台：你也可以使用支持 OpenAI / Anthropic 兼容 API 的平台。

2.2 获取 API Key

1. 访问 <https://api.xi-ai.cn/> 并注册账号
2. 登录后进入控制台
3. 在 API Key 管理页面创建一个新的 API Key
4. 复制生成的 Key（格式类似 `sk-xxxxxxx`）

2.3 配置 Qwen Code

Qwen Code 使用 JSON 格式的配置文件（通常位于 `~/.qwen/settings.json`）。

💡 什么是 `~`？

`~` 是用户主目录的简写符号。在 macOS/Linux 中：

- `~` = `/Users/你的用户名` (macOS)
- `~` = `/home/你的用户名` (Linux)

例如：`~/.qwen/settings.json` 实际路径可能是 `/Users/htc981/.qwen/settings.json`

手动编辑 JSON 文件

打开配置文件：

```
code ~/.qwen/settings.json
```

配置示例 (xi-ai)

```
{
  "env": {
    "XIAI_API_KEY": "sk-your-actual-xiai-key-here"
  },
  "modelProviders": {
    "xiai": [
      {
        "id": "qwen3.5-plus",
        "name": "Qwen 3.5 Plus",
        "baseUrl": "https://api.xi-ai.cn/v1",
        "envKey": "XIAI_API_KEY"
      }
    ]
  }
}
```

配置说明

字段	说明
<code>XIAI_API_KEY</code>	你的 API Key (从 xi-ai 获取)
<code>id</code>	模型名称 (xi-ai 支持 <code>qwen-plus</code> 、 <code>qwen-max</code> 等)
<code>name</code>	模型显示名称 (自定义)
<code>baseUrl</code>	API 地址 (xi-ai 使用 <code>https://api.xi-ai.cn/v1</code>)
<code>envKey</code>	环境变量名称, 对应 "env" 里的内容

 小提示:

- JSON 格式类似于 Python 的字典 (dict), 使用 `key: value` 对, 用逗号分隔。注意: JSON 的 key 必须用双引号, 字符串值也必须用双引号
- 如果设置文件中已有其他内容, 将新的配置接在别的 key 后面 (逗号分隔) 或放在最前面均可, 顺序不影响
- 可以在配置中设置多个模型, Qwen Code 会优先使用第一个, 你也可以后续调整使用的模型
- 如果你用的是其它服务商, 请参考 <https://qwenlm.github.io/qwen-code-docs/en/users/configuration/model-providers/> 进行模型配置

步骤 3: 验证安装

不需要进入 Qwen Code 交互模式, 可以直接在终端执行命令:

```
# 检查版本
qwen --version

# 测试基本功能 (直接在终端执行, 无需进入交互模式)
qwen -p "Hello, world!"
```

如果看到 AI 返回的回复, 说明配置成功!

步骤 4: 配置 Python 环境 (推荐)

注意: 此步骤为可选, 但推荐使用 conda 管理项目环境, 避免依赖冲突。

创建并激活 conda 环境

```
# 进入 todo-app 目录
cd todo-app

# 创建 conda 环境
conda create -n vibe-coding python=3.10 -y

# 激活环境
conda activate vibe-coding
```

安装项目依赖并启动

```
# 安装依赖
pip install -r requirements.txt

# 启动应用
python app.py

# 在浏览器打开 http://localhost:8001 测试
```

为什么使用 conda?

- 隔离项目依赖，避免全局环境污染
- 方便切换不同项目的环境
- 便于团队协作时统一环境

知识介绍

2.1 LLM 基础

什么是 LLM?

LLM = Large Language Model (大型语言模型)

简单来说，LLM 是一个**超级文本预测器**：

- 阅读了互联网上几乎所有文本
- 学习词语之间的关联模式
- 根据上文预测下一个词

重要理解

LLM 是什么	LLM 不是什么
统计预测器	完美的预言家
擅长模式匹配	真正理解含义
快速生成	总是正确
需要上下文	会读心术

关键洞察：给出的上下文越多，预测越准确。

2.2 Prompt Engineering

什么是 Prompt Engineering?

Prompt Engineering 是**设计好的提示词**来获得更好的 AI 输出。

三个核心技巧

(1) 提供示例 (Few-Shot)

示例：
输入: "test@example.com" → 输出: valid
输入: "invalid" → 输出: Invalid

现在判断: "hello@world.com" → ?

(2) 逐步推理 (Chain-of-Thought)

请逐步推理：

1. 首先分析问题
2. 然后列出已知条件
3. 最后得出结论

(3) 结构化指令

角色
你是一名 Python 专家

任务
创建一个函数验证邮箱格式

要求

- 使用正则表达式
- 包含测试用例

Prompt Template (提示词模板)

[Role] 你是一名 [描述]
[Task] 创建/编写 [什么]
[Input] 输入是 [描述]
[Output] 输出应该是 [描述]
[Constraints] 必须 [要求]

提示：这个模板可以帮助你快速构建结构化的提示词，特别适用于复杂任务。

2.3 Vibe Coding

什么是 Vibe Coding?

Vibe Coding = 用自然语言描述你想要什么，让 AI 自己决定如何实现。

Vibe Coding vs Prompt Engineering

Prompt Engineering	Vibe Coding
像"写菜谱"——每步都要说清楚	像"点外卖"——说要什么即可
关注如何问	关注要什么
需要精心设计提示词	自然语言描述意图

为什么现在可以 Vibe Coding? 现代 AI 有了Agent 能力：

- 📅 自己规划步骤
- 🔧 自己使用工具
- 🛠️ 自己检查修复
- ✅ 自己验证结果

迭代循环

描述意图 → AI 生成 → 测试 → 反馈 → AI 修复 → 完成

示例：

1. "创建一个 TODO List 应用" → AI 生成基础代码
2. "添加优先级功能" → AI 添加优先级
3. "用颜色区分优先级" → AI 美化界面

2.4 Harness Engineering

什么是 Harness Engineering?

Harness Engineering 是**让 AI 可靠运行的系统**。

想象马的缰绳——引导马的力量，使其安全工作。

它解决什么问题？

问题	解决方案
AI 会犯错	代码审查、测试
危险操作	权限限制
错误会扩散	自动重试、回滚

最佳实践

1. **始终审查 AI 生成的代码**
2. **运行测试确保正确**
3. **小步迭代，不要一次改太多**
4. **使用版本控制 (git)**

工作坊材料- 可自由改编和分享！